

LayerTRAK: warstwowa analiza atrybucji danych treningowych metodą TRAK

Mateusz Ostaszewski¹, Natalia Pieczko¹, and Michał Sadowski¹

Wyjaśnialna Sztuczna Inteligencja
Politechnika Warszawska

Abstract. Projekt LayerTRAK bada, jak ograniczenie metody TRAK do wybranych warstw sieci neuronowej wpływa na jakość atrybucji danych treningowych. Eksperymenty prowadzone są na zbiorze CIFAR-10 dla architektur ResNet-18, ResNet-34 oraz MobileNetV2. Dla każdej architektury porównywane są konfiguracje obejmujące wyłącznie głowę klasyfikacyjną, późne warstwy, warstwy środkowo-późne, wczesne warstwy oraz cały model. Jako główna metryka jakości używana jest LDS, czyli Linear Datamodeling Score, obliczana z wykorzystaniem modeli pomocniczych trenowanych na losowych podzbiorach danych.

Keywords: data attribution · TRAK · LDS · CIFAR-10 · ResNet · MobileNetV2

1 Wprowadzenie

Współczesne modele głębokiego uczenia osiągają wysoką skuteczność dzięki uczeniu na dużych zbiorach danych, ale ich zachowanie jest trudne do wyjaśnienia na poziomie pojedynczych przykładów treningowych. Metody atrybucji danych treningowych próbują odpowiedzieć na pytanie, które obserwacje ze zbioru uczącego miały największy wpływ na konkretną predykcję modelu.

Celem projektu jest zbadanie, czy metoda TRAK [2] może być stosowana nie tylko dla całego modelu, ale również dla wybranych zakresów parametrów. Takie podejście może zmniejszyć koszt obliczeń i jednocześnie pokazać, w których częściach sieci znajduje się najbardziej użyteczny sygnał atrybucji.

1.1 Cel projektu

Głównym celem jest porównanie jakości atrybucji uzyskanej metodą TRAK dla różnych konfiguracji warstw i architektur. Projekt odpowiada na trzy pytania:

- Czy późne warstwy modelu wystarczają do uzyskania jakości atrybucji zbliżonej do pełnego modelu?
- Jak wybór zakresu warstw wpływa na stabilność numeryczną i rozkład wyników TRAK?
- Czy zależności te są podobne dla architektur ResNet-18, ResNet-34 i MobileNetV2?

1.2 Zakres pracy

Zakres obejmuje przygotowanie pipeline'u danych dla CIFAR-10, trening modeli bazowych, implementację selektywnego wyboru parametrów w TRAK, trening ensemble'u modeli pomocniczych dla LDS oraz analizę porównawczą wyników.

2 Podstawy Teoretyczne

2.1 Atrybucja Danych Treningowych

Atrybucja danych treningowych przypisuje każdej próbce uczącej wartość opisującą jej wpływ na wybrane zachowanie modelu, np. logit klasy predykowanej dla konkretnego przykładu testowego. Dodatnia wartość wpływu oznacza, że obecność próbki wspiera daną predykcję, natomiast wartość ujemna wskazuje, że próbka działa w przeciwnym kierunku. Taki sposób myślenia jest bliski podejściu datamodels, w którym analizuje się zmiany predykcji modelu pod wpływem zmian składu zbioru treningowego [1].

2.2 TRAK

TRAK, czyli Tracing with the Randomly-Projected After Kernel, jest metodą atrybucji skalującą klasyczne podejścia gradientowe przez lokalną linearyzację modelu oraz losowe rzutowania gradientów [2]. Dla przykładu treningowego z_i i przykładu testowego z_t metoda wyznacza wynik wpływu na podstawie podobieństwa reprezentacji gradientowych względem parametrów modelu.

W projekcie TRAK został opakowany w moduł `LayerTRAKRunner`, który umożliwia przekazywanie listy parametrów `grad_wrt`. Dzięki temu ten sam algorytm można uruchamiać dla całego modelu albo tylko dla wybranego prefiksu nazw parametrów. Implementacja korzysta z oficjalnej biblioteki TRAK jako punktu wyjścia [3].

2.3 LDS

Linear Datamodeling Score (LDS) służy do ilościowej oceny jakości atrybucji. Intuicja LDS jest następująca: jeżeli score'y atrybucji dobrze opisują wpływ danych, to suma score'ów dla próbek obecnych w danym podzbiorze treningowym powinna dobrze przewidywać zachowanie modelu wytrenowanego właśnie na tym podzbiorze [1, 2].

Dla maski podzbioru $m \in \{0, 1\}^n$ i wektora wpływów s_t dla ustalonego przykładu testowego t przewidywanie liniowe można zapisać jako:

$$\hat{y}_{t,m} = \sum_{i=1}^n m_i s_{t,i}. \quad (1)$$

LDS porównuje takie przewidywania z rzeczywistymi wynikami modeli pomocniczych trenowanych na odpowiadających im podzbiórach. W praktyce porównanie wykonywane jest za pomocą korelacji rang Spearmana.

3 Metodyka Eksperymentalna

3.1 Zbiór Danych

Eksperymenty prowadzone są na CIFAR-10 [4], który zawiera 50 000 przykładów treningowych i 10 000 przykładów testowych reprezentujących 10 klas. Obrazy mają rozmiar 32×32 piksele i trzy kanały koloru. W treningu stosowane są standardowe augmentacje: losowe przycięcie z paddingiem, losowe odbicie poziome, konwersja do tensora oraz normalizacja statystykami CIFAR-10. W celu oceny TRAK wykorzystano podzbiór obejmujący pierwsze 1000 przykładów testowych.

3.2 Architektury Modeli

Porównywane są trzy architektury:

- ResNet-18 jako podstawowa architektura rezydualna,
- ResNet-34 jako głębszy wariant z tym samym typem bloków,
- MobileNetV2 jako lżejsza architektura z inną strukturą bloków.

Modele tworzone są przez moduł `model_factory` i wykorzystują wagi wstępnie trenowane na ImageNet. Trening modeli bazowych jest fine-tuningiem: przy domyślnym ustawieniu `freeze_early=True` aktualizowane są końcowe bloki i głowa klasyfikacyjna, a wcześniejsze warstwy pozostają zamrożone. Dla ResNetów trenowane są warstwy `layer4` oraz `fc`, natomiast dla MobileNetV2 bloki `features[14:19]` oraz `classifier`. Modele bazowe trenowane są przez 30 epok z batch size równym 128. Tabela 1 zawiera finalne wyniki skuteczności tych modeli.

Wszystkie obliczenia eksperymentalne, w tym trening modeli bazowych, modeli pomocniczych oraz estymacja atrybucji metodą TRAK, zostały przeprowadzone z wykorzystaniem karty graficznej NVIDIA GeForce RTX 3080 (10 GB VRAM).

Table 1. Skuteczność modeli bazowych.

Architektura	Liczba epok	Accuracy test $\uparrow$
ResNet-18	30	0.7020
ResNet-34	30	0.6698
MobileNetV2	30	0.6882

3.3 Konfiguracje Warstw

Każda architektura jest oceniana w pięciu konfiguracjach. Dla ResNetów wybór warstw opiera się na prefiksach `conv1`, `layer1`, `layer3`, `layer4` i `fc`. Dla MobileNetV2 wybór opiera się na zakresach bloków `features` oraz klasyfikatorze. Pełne mapowanie konfiguracji przedstawia tabela 2.

Table 2. Badane konfiguracje warstw.

Konfiguracja	ResNet-18 / ResNet-34	MobileNetV2
head_only	fc	classifier
late	layer4 + fc	features[14:19] + classifier
mid_late	layer3	features[7:14]
early	conv1 + layer1	features[0:7]
full_model	wszystkie parametry	wszystkie parametry

3.4 Pipeline TRAK

Zgodnie z literaturą, aby wygładzić szum estymacji metody TRAK, dla każdego modelu bazowego pobrano 5 ostatnich checkpointów (epoki 26-30). Proces ekstrakcji cech oraz ewaluacji atrybucji danych przeprowadzono osobno dla każdego z nich, a następnie ostateczna macierz score'ów (używana do liczenia LDS) została wyliczona jako średnia arytmetyczna z tych pięciu macierzy.

Pipeline eksperymentalny składa się z następujących etapów:

1. wczytanie checkpointu modelu bazowego,
2. wybór listy parametrów odpowiadającej konfiguracji warstw,
3. wyznaczenie reprezentacji gradientowych dla całego zbioru treningowego CIFAR-10,
4. obliczenie punktacji wpływu (ang. *scores*) dla 1000 przykładów testowych względem 50 000 przykładów treningowych dla każdego z 5 ostatnich checkpointów (epoki 26-30) niezależnie,
5. zapis macierzy wyników, statystyk i metadanych eksperymentu.

W implementacji używany jest wymiar losowego rzutowania równy 1024 oraz regularyzacja $\lambda = 10^{-3}$. Dodatkowo użyto wariantu `RobustScoreComputer`, który w przypadku osobliwej macierzy przechodzi z odwracania macierzy na pseudo-odwrotność.

3.5 Pipeline LDS

Do ilościowej oceny jakości atrybucji wykorzystywany jest wskaźnik LDS. Proces ten został zaimplementowany w module `lds` i składa się z trzech kluczowych etapów:

1. *Generowanie masek i trening ensemble.* Dla każdej z trzech architektur wytrenowano ensemble składający się z 40 modeli pomocniczych. Każdy model był trenowany na losowym podzbiórze stanowiącym 50% oryginalnego zbioru treningowego CIFAR-10. Maski podzbiorów $m \in \{0,1\}^n$ są generowane niezależnie, co zapewnia różnorodność danych widzianych przez poszczególne modele.

2. *Wyznaczanie marginesu pewności.* Zgodnie z implementacją w skrypcie `compute_lds.py`, jako funkcję celu do wyliczania LDS zastosowano margines pewności, zdefiniowany jako różnica między logitem poprawnej klasy a najwyższym logitem klasy błędnej. Wartość ta pozwala płynnie mierzyć pewność modelu względem poprawnej odpowiedzi.
3. *Obliczanie korelacji.* Wartość LDS dla ustalonego przykładu testowego obliczana jest jako korelacja rang Spearmana pomiędzy rzeczywistymi marginesami pewności uzyskanymi z ensemble'u modeli oraz przewidywanymi marginesami uzyskanymi przez sumowanie wyników TRAK dla przykładów treningowych obecnych w danej masce. Finalny wynik LDS prezentowany w tabelach jest średnią wartością korelacji po wszystkich 1000 sprawdzanych przykładach testowych.

4 Wyniki

Ta sekcja zawiera wyniki dla ostatnich pięciu checkpointów modeli bazowych. Wizualizacje i statystyki wygenerowano przy użyciu projektowych skryptów `plot_trak_results` oraz `analize_trak_results`.

4.1 Porównanie LDS

Tabela 3 zawiera wartości LDS uśrednione po pięciu epokach dla par architektura-konfiguracja. Tabela 4 pokazuje ten sam snapshot zagregowany po architekturach.

Table 3. Porównanie LDS uśrednione po pięciu epokach.

Architektura	Head ↑	Late ↑	Mid+Late ↑	Early ↑	Full ↑
ResNet-18	0.0863	0.0582	0.0329	0.0138	0.0445
ResNet-34	0.0910	0.0567	0.0209	0.0026	0.0342
MobileNetV2	0.0953	0.0703	0.0316	0.0064	0.0368

W uśrednionym snapshotcie najwyższy LDS dla każdej architektury uzyskuje konfiguracja `head_only`. Drugą najmocniejszą konfiguracją jest konfiguracja `late`. Konfiguracje `mid_late` i `early` osiągają wyraźnie niższe wartości.

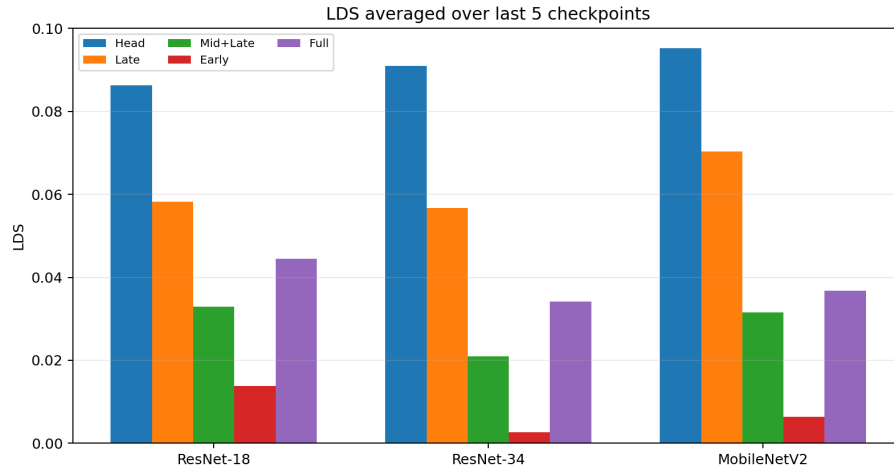
Rysunek 1 pokazuje te same wartości co tabela 3, ale w formie ułatwiającej porównanie konfiguracji wewnątrz każdej architektury.

4.2 Rozkłady Score'ów TRAK

Oprócz LDS przeanalizowano statystyki rozkładów score'ów TRAK: średnią, odchylenie standardowe, percentyle, wartości ekstremalne oraz średni najsilniejszy pozytywny wpływ dla przykładu testowego. Tabela 5 przedstawia średnie statystyki po konfiguracjach warstw na podstawie wyników wygenerowanych

Table 4. Średni LDS po architekturach dla wyników uśrednionych po pięciu epokach.

Konfiguracja	Średni LDS ↑
head_only	0.0909
late	0.0618
full_model	0.0385
mid_late	0.0286
early	0.0076

**Fig. 1.** Porównanie LDS dla architektur i konfiguracji warstw.**Table 5.** Średnie statystyki rozkładów TRAK po konfiguracjach warstw.

Konfiguracja	Liczba	Mean std ↓	Mean P99 abs ↓	Mean top-1 ↑
head_only	3	5.704	21.180	102.566
late	3	1.447	5.186	15.938
mid_late	3	7.336	26.151	67.071
early	3	76.561	246.255	2272.553
full_model	3	4.602	16.120	41.595

przez skrypt wizualizacji. Metryka **top-1** opisuje siłę najsilniejszego dodatniego wpływu, a nie samodzielną jakość konfiguracji.

Najbardziej stabilną konfiguracją pod względem rozrzutu score'ów jest **late**. Ma ona najniższe średnie odchylenie standardowe oraz najniższy średni 99. percentyl wartości bezwzględnych. Konfiguracje **mid_late** i **early** mają znacznie szersze rozkłady, co wskazuje na większą niestabilność score'ów. Wysokie wartości dla **early** są zdominowane przez **mobilenetv2/early**; jego rozrzut jest znacznie większy niż dla ResNetów. **head_only** osiąga wysokie wartości top-1, ale jednocześnie generuje silniejsze ogony niż **late**.

Table 6. Porównanie konfiguracji **late** między architektuрами.

Architektura	Std ↓	P99 ↓	P99 abs ↓	Top-1 mean ↑
ResNet-18	1.534	4.598	5.625	16.404
ResNet-34	1.245	3.613	4.418	15.680
MobileNetV2	1.562	4.535	5.516	15.729

Tabela 6 pokazuje, że **late** jest porównywalne między wszystkimi trzema architektuрами. Najwęższy rozkład w tej konfiguracji ma ResNet-34, natomiast MobileNetV2 ma rozrzut zbliżony do ResNet-18. Wartości **top1_mean** są bardzo podobne dla wszystkich trzech modeli.

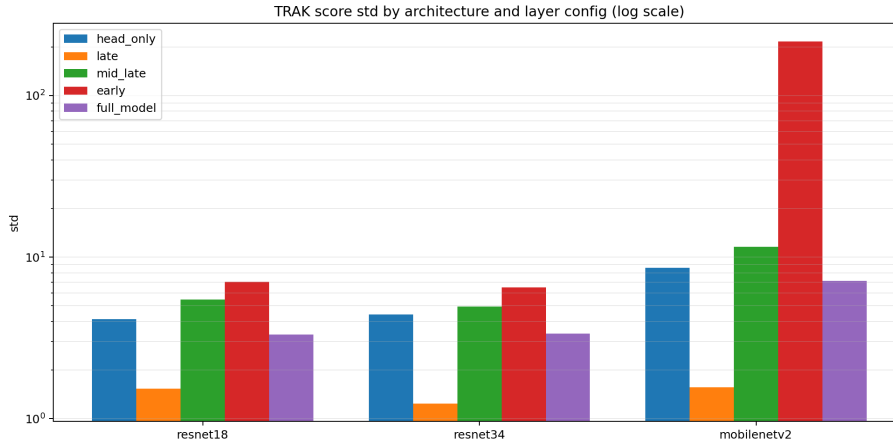


Fig. 2. Odchylenie standardowe score'ów TRAK dla konfiguracji warstw w skali logarytmicznej.

Rysunki 2 i 3 zostały wygenerowane na podstawie zestawu wyników z ostatnich pięciu checkpointów.

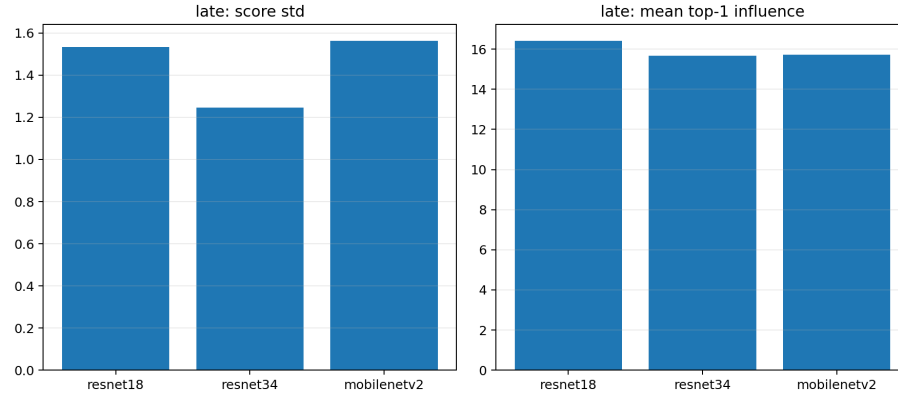


Fig. 3. Porównanie stabilnej konfiguracji **late** między architekturami.

4.3 Przykłady Jakościowe Atrybucji

Dla wybranych targetów wygenerowano również siatki przykładów treningowych o największym dodatnim i ujemnym wpływie. Każda siatka pokazuje target po lewej stronie, pięć przykładów o największym dodatnim score pośrodku oraz pięć przykładów o najbardziej ujemnym score w dolnym wierszu. Takie wizualizacje nie zastępują LDS, ale pomagają sprawdzić, czy score'y TRAK są semantycznie interpretowalne.

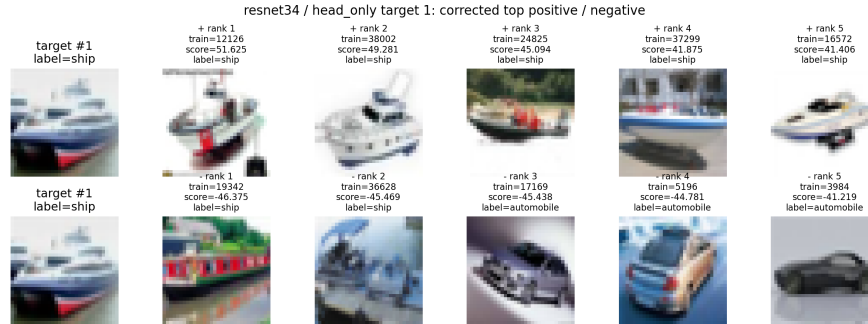


Fig. 4. Przykład jakościowy dla ResNet-34 i konfiguracji **head_only**. Target należy do klasy **ship**; dodatnie przykłady są głównie statkami, a ujemne obejmują klasy konkurencyjne, m.in. **airplane** i **automobile**.

Rysunki 4 i 5 pokazują dwie perspektywy interpretacji. **head_only** daje najmocniejszy uśredniony wynik LDS, a **late** jest najbardziej stabilną konfiguracją pod względem rozrzutu score'ów. W obu przypadkach dodatnie przykłady często

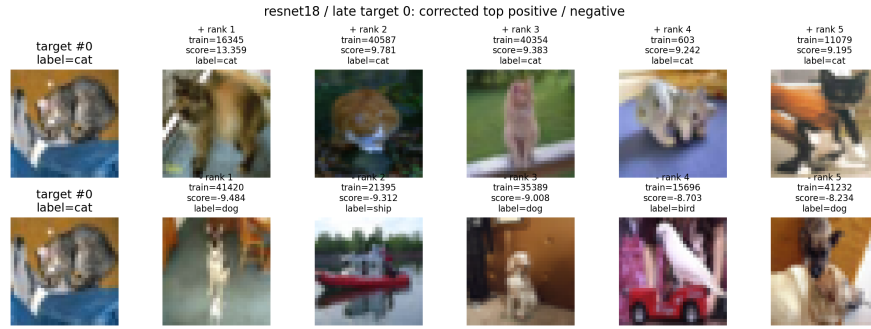


Fig. 5. Przykład jakościowy dla ResNet-18 i konfiguracji *late*. Target należy do klasy *cat*; dodatkowo przykłady zawierają podobne wizualnie obrazy, natomiast ujemne wskazują przypadki konkurencyjne lub mylące dla reprezentacji końcowych warstw.

są zgodne semantycznie z targetem, natomiast przykłady ujemne mogą wskazywać klasy lub obrazy, które działają w przeciwnym kierunku dla analizowanej predykcji.

5 Dyskusja

Na podstawie przeprowadzonych eksperymentów i wyliczonych wartości LDS można stwierdzić, że ograniczenie algorytmu TRAK do późnych warstw sieci nie tylko wystarcza do zachowania jakości atrybucji, ale wręcz ją poprawia. We wszystkich trzech badanych architekturach (ResNet-18, ResNet-34 oraz MobileNetV2) najwyższą zgodność z empirycznymi modelami pomocniczymi (LDS) osiągnęła konfiguracja *head_only*. Drugą najbardziej wartościową konfiguracją okazała się *late*, obejmująca ostatnie bloki konwolucyjne i klasyfikator. Najwyższy wynik LDS wynoszący 0.0953 osiągnął model MobileNetV2 dla konfiguracji *head_only*.

Wyniki te stanowią mocny argument za stosowaniem podejścia warstwowego (LayerTRAK). Analiza pełnego modelu (*full_model*) osiągała gorsze wyniki niż konfiguracje końcowe, co dowodzi, że włączanie wczesnych parametrów sieci do obliczeń atrybucyjnych wprowadza szum informacyjny i degradowa jakość przewidywań metody TRAK.

5.1 Uzasadnienie teoretyczne: Linearyzacja a głębokość warstw

Uzyskane wyniki są spójne z matematycznymi fundamentami algorytmu TRAK. Metoda ta opiera się na lokalnej linearyzacji modelu w celu estymacji wpływu danych. Głowa klasyfikacyjna (*head_only*) to z definicji operacja liniowa (mnożenie macierzy wag i dodanie biasu), co sprawia, że przybliżenie wykorzystywane przez TRAK jest w tym miejscu najbardziej precyzyjne.

Z kolei głębokie, wczesne warstwy sieci (**early**, **mid_late**) charakteryzują się wysoką nieliniowością oraz odpowiadają za ekstrakcję niskopoziomowych cech (np. krawędzi czy tekstur). Usunięcie pojedynczego zdjęcia treningowego ze zbioru nie zmienia drastycznie sposobu, w jaki sieć rozpoznaje krawędzie, co tłumaczy wyniki oscylujące wokół zera dla konfiguracji **early**. Algorytm TRAK, poszukujący liniowych zależności, zawodzi w obliczu głębokich nieliniowości, przez co jego predykcje dla wczesnych warstw przypominają losowy szum.

5.2 Różnice i podobieństwa między architekturami

Zaobserwowana hierarchia przydatności warstw (**head_only** > **late** > **full_model** > **mid_late** > **early**) pozostaje stabilna niezależnie od architektury. Zarówno podstawowy ResNet-18, głębszy ResNet-34, jak i MobileNetV2 wykazują niemal identyczne trendy spadku wartości metryki LDS wraz z zagłębianiem się w początkowe warstwy sieci.

Istotnym problemem okazała się numeryczna wrażliwość modelu MobileNetV2. Zastosowane w jego architekturze wąskie gardła generują skrajne wartości gradientów we wczesnych warstwach, co destabilizuje kluczowe dla algorytmu TRAK odwracanie macierzy. W trakcie eksperymentów analiza pojedynczego punktu kontrolnego (np. tylko dla ostatniej, 30. epoki) w konfiguracji **early** wielokrotnie kończyła się wynikiem NaN. Otrzymanie stabilnych wyników było możliwe dopiero po zastosowaniu uśredniania z pięciu epok, które wygładziło drastyczne skoki wartości. Zjawisko to dowodzi, że MobileNetV2 stanowi znacznie większe wyzwanie numeryczne w analizie atrybucyjnej niż architektury z rodziny ResNet.

5.3 Podsumowanie

Ograniczenie atrybucji danych do samej głowy klasyfikacyjnej lub ostatnich bloków funkcyjnych sieci jest optymalnym kompromisem. Pozwala to nie tylko na znaczne zmniejszenie kosztów obliczeniowych i zapotrzebowania na pamięć VRAM (śledzenie tysięcy zamiast milionów parametrów), ale przede wszystkim dostarcza najwyższej i najbardziej rzetelnej korelacji (LDS) z rzeczywistym zachowaniem modelu.

6 Ograniczenia

Najważniejsze ograniczenia projektu:

- eksperymenty dotyczą jednego zbioru danych, CIFAR-10,
- liczba modeli pomocniczych LDS jest ograniczona kosztem treningu,
- LDS zależy od jakości i liczby losowych masek podzbiorów,
- TRAK jest wrażliwy na kondycję macierzy gradientowej i wybór parametrów rzutowania,
- wnioski dla MobileNetV2 mogą wymagać osobnej diagnostyki ze względu na inną strukturę architektury i zamrożone warstwy.

7 Instrukcja Uruchomienia

Repozytorium używa `uv` oraz `make`. Podstawowy scenariusz uruchomienia:

1. instalacja zależności: `make install`,
2. trening modeli bazowych: `make train`,
3. generowanie masek LDS: `make masks`,
4. trening modeli ensemble: `make ensemble-train-all`,
5. wyodrębnienie logitów z modeli ensemble: `make extract-ensemble-all`,
6. uruchomienie TRAK: `make experiment`,
7. obliczenie wartości LDS: `make compute-lds`,
8. generowanie wizualizacji: `make visualize-trak`.

8 Wnioski

Przeprowadzone eksperymenty jednoznacznie pokazują, że warstwowe podejście do atrybucji danych (LayerTRAK) ma sens praktyczny. Ograniczenie algorytmu TRAK do wybranych, końcowych fragmentów sieci nie tylko zmniejsza zapotrzebowanie na pamięć i moc obliczeniową, ale również zauważalnie poprawia jakość samej atrybucji w porównaniu do analizy pełnego modelu.

Najlepszym wyborem i optymalnym kompromisem okazała się analiza wyłącznie głowy klasyfikacyjnej (`head_only`). Konfiguracja ta oferuje najwyższą zgodność z empirycznymi modelami pomocniczymi (LDS) przy absolutnie minimalnym koszcie obliczeniowym, co czyni ją idealną do szybkich zastosowań praktycznych. Z kolei konfiguracja `late` stanowi doskonałą, wysoce stabilną numeryczną alternatywę w przypadkach, gdy zależy nam na śledzeniu głębszych cech semantycznych obrazu. Wykazano jednoznacznie, że włączanie wczesnych warstw sieci (`early`) do algorytmu TRAK jest nieefektywne i wprowadza jedynie szum.

W celu pełniejszego zrozumienia zbadanego zjawiska, w ramach dalszych prac badawczych warto rozważyć następujące eksperymenty:

- weryfikacja podejścia warstwowego na większych i bardziej złożonych zbiorach danych (np. ImageNet) oraz w innych domenach (np. modele językowe NLP),
- głębsza analiza problemów numerycznych we wczesnych warstwach modeli mobilnych (MobileNetV2),
- zbadanie wpływu liczby użytych punktów kontrolnych w technice *Single Trajectory Ensembling* na stabilność metryki LDS, w celu wyznaczenia idealnego balansu między jakością a czasem obliczeń.

References

1. Ilyas, A., Park, S.M., Engstrom, L., Leclerc, G., Madry, A.: Datamodels: Predicting Predictions from Training Data. <https://arxiv.org/abs/2202.00622> (2022)

2. Park, S.M., Georgiev, K., Ilyas, A., Leclerc, G., Madry, A.: TRAK: Attributing Model Behavior at Scale. In: International Conference on Machine Learning (ICML) (2023). <https://arxiv.org/abs/2303.14186>
3. MadryLab: TRAK official implementation. <https://github.com/MadryLab/trak>
4. Krizhevsky, A.: Learning Multiple Layers of Features from Tiny Images. Technical report, University of Toronto (2009)